CookBook

Project Report

A Recipe Sharing & Browsing Web Application built with React.js and Node.js

INTRODUCTION

CookBook is a versatile and user-friendly web application designed to streamline the discovery, creation, and sharing of recipes online. Suitable for home cooks, food bloggers, culinary students, and everyday cooking enthusiasts, it offers a wide range of features — including recipe creation, ingredient listing, step-by-step cooking instructions, category-based browsing, and community-driven reviews — all accessible through a clean and responsive interface.

Users can fully personalize their cooking profiles, save their favourite recipes to a wishlist, and share their own culinary creations with a growing community of food lovers. Robust security features such as JWT-based authentication, SSL encryption, and role-based access control help protect user data and ensure platform integrity. CookBook also supports smooth admin workflows, enabling administrators to manage content, users, and platform-wide activity with ease.

With built-in data visualization tools on the admin dashboard, platform managers can monitor usage trends and generate activity reports for deeper analysis. Its flexibility, ease of use, and powerful feature set make CookBook an ideal solution for individuals and communities seeking an efficient, engaging digital recipe-management platform.

SCENARIO BASED CASE STUDY

Meet Arjun, a passionate home cook and food enthusiast with a busy daily schedule who values convenience and culinary inspiration in his life. Arjun loves experimenting with new dishes in his kitchen but often struggles to organize his growing collection of recipes and discover new ones that match his taste preferences and available ingredients.

Arjun discovers CookBook, a comprehensive web application designed to make recipe sharing and browsing effortless. Intrigued by its features, Arjun decides to explore how CookBook can help him organize his culinary world and connect with a community of fellow food lovers.

-

- **User Registration and Authentication:** Arjun registers an account on CookBook, providing his name, email, and cooking preferences. He logs in securely using his credentials, ensuring that his personal recipe collection and data remain private and secure.
- **Recipe Browsing:** Arjun explores CookBook's extensive recipe catalog, organized by categories such as Breakfast, Lunch, Dinner, Desserts, and Beverages. He filters recipes by cuisine type, preparation time, difficulty level, and dietary requirements to quickly find what suits him.
- **Recipe Creation and Sharing:** Arjun creates and publishes his own recipes on the platform, adding a title, list of ingredients, step-by-step instructions, preparation time, difficulty rating, and a photo of the finished dish. He shares it with the community and receives feedback from other users.
- **Wishlist and Favourites:** Arjun saves recipes he wants to try later to his personal wishlist, making it easy to revisit them without searching again. His wishlist acts as a personalized digital recipe book.
- **Reviews and Ratings:** After trying a recipe from another user, Arjun leaves a rating and a written review, sharing his experience and suggestions. This helps other users make informed choices and helps recipe authors improve their content.
- **Admin-Managed Platform:** Behind the scenes, the platform admin ensures that content is appropriate, user accounts are properly managed, and the overall platform runs smoothly. The admin monitors all activity through a dedicated dashboard with charts and usage reports.

Thanks to CookBook, Arjun can now organize and share his culinary passion more efficiently and enjoyably. He has built connections with fellow food lovers, published several popular recipes, and discovered hundreds of new dishes to try. CookBook has become an indispensable part of Arjun's daily routine, helping him achieve his culinary goals with ease and confidence.

SYSTEM REQUIREMENTS

To ensure smooth development, deployment, and usage of the CookBook Web Application, certain system prerequisites must be met. These requirements are categorized into software, setup, and hardware specifications, all of which contribute to building a robust and scalable recipe-sharing platform.

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

Component	Specification	Purpose
Operating System	Windows 10/11, macOS, or Linux	Supports cross-platform development and testing
Node.js	v16 or above	Runtime environment for backend logic and API routing
npm	v8 or above	Package manager for React and all related libraries
React.js	Latest Stable	JavaScript library for dynamic and responsive user interfaces
Express.js	Latest Stable	Lightweight web framework for building RESTful APIs
MongoDB	v6 or above	NoSQL database for storing users, recipes, and wishlist data
Browser	Chrome / Firefox (latest)	For rendering and testing the UI in real-time
Postman	Latest	Tool for testing APIs during development
Visual Studio Code	Latest	Preferred code editor with built-in Git and terminal support

2. Hardware Requirements

Describes the minimum and recommended specifications needed to support the development and usage of the application.

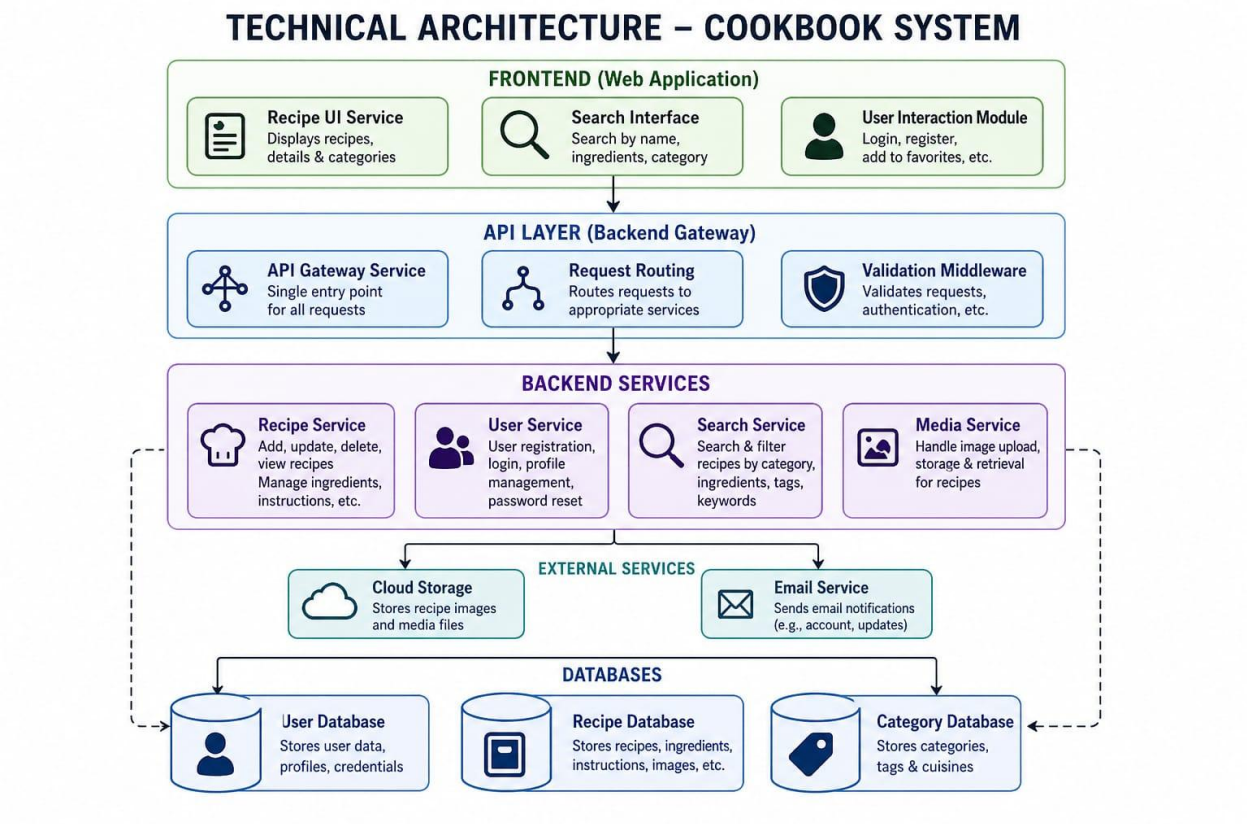
Component	Minimum Specification	Recommended
Processor	Intel Core i5 (8th Gen) / AMD Ryzen 5	Intel Core i7 / AMD Ryzen 7 or better
RAM	8 GB	16 GB — for handling dev servers, IDEs, and browser testing
Storage	1 GB free space	5 GB — for packages, MongoDB setup, and project files
Display	1366x768 resolution	1920x1080 or higher for optimal coding and layout visualization

PROJECT ARCHITECTURE

The CookBook project follows a standard full-stack MERN architecture with a clear separation between the client-side (Frontend) and server-side (Backend) layers. The architecture is structured as follows:

Layer	Technology	Role
Frontend	React.js (Vite)	Handles UI, user interactions, and REST API communication
Backend	Node.js + Express.js	Processes requests, applies business logic, and interfaces with the database
Database	MongoDB + Mongoose	Stores users, recipes, reviews, and wishlist data
Authentication	JWT (JSON Web Tokens)	Secures routes and manages sessions for User and Admin roles
File Storage	Multer	Handles uploading and storing recipe images and profile pictures
API Testing	Postman	Used during development to test and validate all API endpoints

TECHNICAL ARCHITECTURE:



In this architecture, the flow begins with a user interacting with the CookBook frontend built in React.js. The user either browses recipes, submits a new recipe, manages their wishlist, or performs account-related actions through the frontend interface.

The CookBook frontend then sends a request to the CookBook backend built with Node.js and Express.js. The backend processes the request and the associated data — whether it is a new recipe submission, a search query, a wishlist update, or a user login attempt.

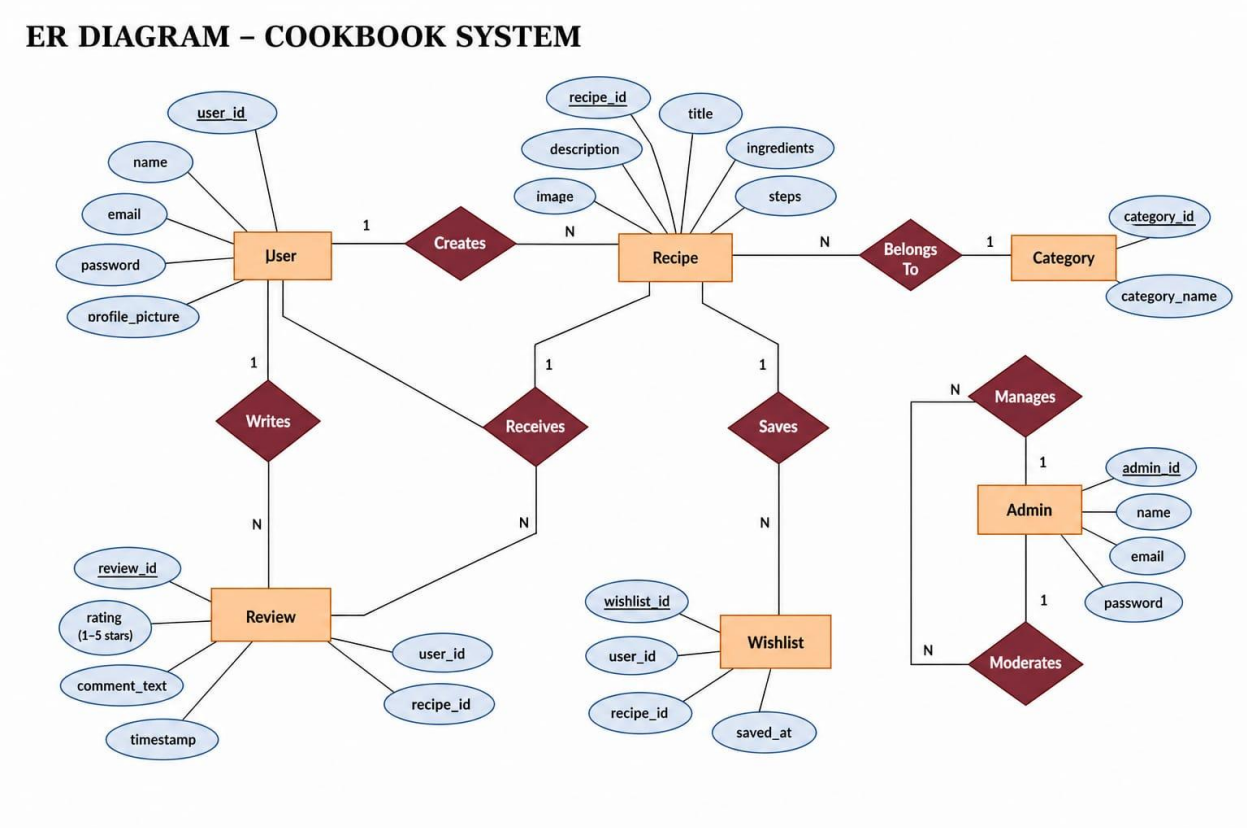
The backend communicates with a MongoDB database to store and retrieve recipes, user accounts, reviews, and wishlist data securely. The database stores all submitted data including user profiles, recipe details, ingredient lists, step-by-step instructions, and ratings.

Additionally, the backend utilizes a file storage system to store images associated with recipes and user profiles, including uploaded dish photographs and profile pictures relevant to the platform.

Once the backend has processed the request, it sends a response back to the CookBook frontend. The frontend then displays the result to the user — whether it is a recipe listing, a confirmation message, or a dashboard update.

The Admin interacts through a separate admin panel, which connects to the same backend with elevated permissions, allowing full control over user accounts, recipe content, and platform analytics.

ER DIAGRAM



The Entity-Relationship (ER) diagram for CookBook represents the various entities and their relationships within the recipe sharing and management system. Here is a description of the key components of the ER diagram:

1. Entities

- Recipe: Represents dishes and culinary creations available on the platform, with attributes such as recipe ID, title, description, ingredients, steps, cooking time, difficulty level, category, and image.
- User: Represents registered members of the platform, with attributes such as user ID, name, email address, password, and profile picture.
- Admin: Represents platform administrators with attributes such as admin ID, name, email, and password. Admins have elevated permissions to manage all platform content and users.
- Category: Represents recipe categories such as Breakfast, Lunch, Dinner, Desserts, Snacks, and Beverages, with attributes such as category ID and category name.
- Review: Represents user feedback on recipes, with attributes such as review ID, rating (1–5 stars), comment text, and timestamp.
- Wishlist: Represents a user's saved or favourite recipes, acting as a join entity between User and Recipe.

2. Relationships

- User — Recipe (Creates): A user can create multiple recipes; each recipe is authored by one user. This is a one-to-many relationship.
- Recipe — Category (Belongs To): Each recipe belongs to one category; a category can contain many recipes. This is a many-to-one relationship.
- User — Review (Writes): A user can write multiple reviews; each review is written by one user for one specific recipe.
- Recipe — Review (Receives): A recipe can receive multiple reviews from different users. This is a one-to-many relationship.
- User — Wishlist — Recipe (Saves): A user can save multiple recipes to their wishlist; a recipe can be saved by multiple users. Wishlist acts as a many-to-many join entity.
- Admin — User (Manages): The admin can view and manage all registered user accounts on the platform.
- Admin — Recipe (Moderates): The admin can review, edit, or remove any recipe from the platform.

3. Attributes

- Each entity has its own set of attributes that describe its properties. For example, the Recipe entity has attributes such as recipe ID, title, ingredients list, preparation steps, cooking time, and category.

4. Primary Keys

- Each entity has a primary key that uniquely identifies each record in the entity. For example, the Recipe entity's primary key is the recipe ID; the User entity's primary key is the user ID.

5. Foreign Keys

- Foreign keys are used to establish relationships between entities. For example, the Recipe entity contains a foreign key referencing the user ID of its author, and the Review entity contains foreign keys referencing both the user ID and the recipe ID.

Overall, the ER diagram for CookBook provides a visual representation of the database schema, showing how different entities are related and how data flows between them in the recipe sharing and browsing platform.

KEY FEATURES:

The key features of the CookBook web application are designed to deliver an intuitive and engaging experience for home cooks and food enthusiasts. Here are the core features included in the platform:

- **User Registration and Profiles:** Allow users to create accounts, set up personal cooking profiles, upload profile pictures, and manage their account information and dietary preferences.
- **Recipe Browsing and Discovery:** Allow users to search and browse a wide collection of recipes, filtering by category, cuisine type, preparation time, difficulty level, and dietary requirements such as vegan or gluten-free.
- **Recipe Creation and Sharing:** Enable users to publish their own recipes with a title, ingredient list, step-by-step cooking instructions, preparation time, difficulty rating, and an uploaded dish photo.
- **Category-Based Navigation:** Organize all recipes into well-defined categories such as Breakfast, Lunch, Dinner, Desserts, Snacks, and Beverages, making discovery simple and intuitive.
- **Reviews and Ratings:** Enable users to rate recipes on a 1–5 star scale and write detailed comments, helping the community make informed cooking decisions.
- **Wishlist / Favourites:** Allow users to save their favourite recipes to a personal wishlist for quick access later, acting as a personalized digital recipe collection.
- **My Recipes:** Maintain a dedicated page where users can view, edit, and manage all the recipes they have personally created and published on the platform.

- **Admin Dashboard:** Provide the administrator with a dedicated panel to manage all user accounts, moderate recipe content, view platform statistics through visual charts, and ensure smooth operation of the platform.

ROLES AND RESPONSIBILITIES

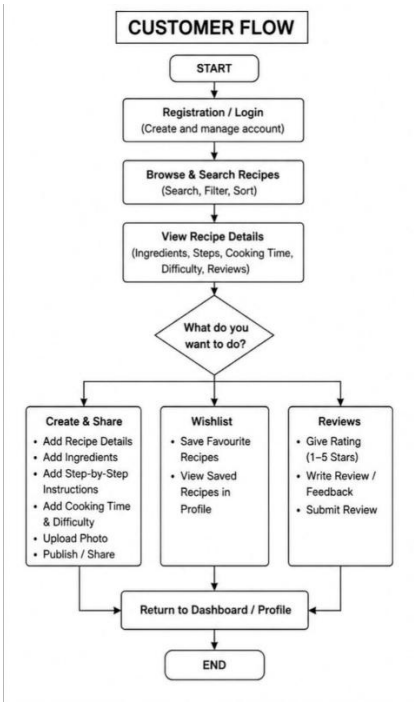
User:

- Registration and Account Management: Users are responsible for creating an account, providing accurate personal information, and managing their profile details including profile picture and password security.
- Recipe Browsing and Search: Users can search and browse all available recipes based on their preferences, including filtering by category, cuisine type, preparation time, and difficulty level.
- Recipe Creation: Users can create and publish their own recipes by providing a title, ingredient list, cooking steps, preparation time, difficulty level, category, and a photo of the dish.
- Reviews and Ratings: Users are responsible for providing honest and constructive reviews and ratings on recipes they have tried, helping others make informed decisions and helping authors improve their content.
- Wishlist Management: Users can add recipes to their personal wishlist and manage their saved collection from their profile dashboard.
- Communication: Users may contact the platform's support team for any queries, issues, or feedback regarding their account or experience on the platform.

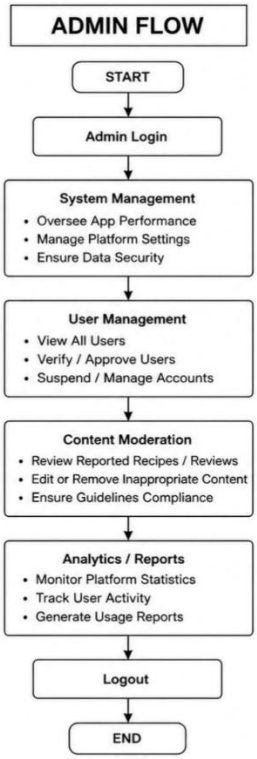
Admin:

- System Management: The admin is responsible for the overall management and administration of the CookBook platform, including monitoring system performance, ensuring data security and privacy, and managing application settings and configurations.
- User Management: The admin manages all user accounts, including registration oversight, account verification, and account moderation — handling approvals, suspensions, or terminations when necessary.
- Content Moderation: The admin oversees all published recipes and user-submitted reviews to ensure that content meets community standards, is accurate, and is free from inappropriate material.
- Analytics and Reporting: The admin has access to a dashboard with visual charts and statistics on platform usage, including number of registered users, total recipes published, popular categories, and recent activity.

User Flow:



Admin:



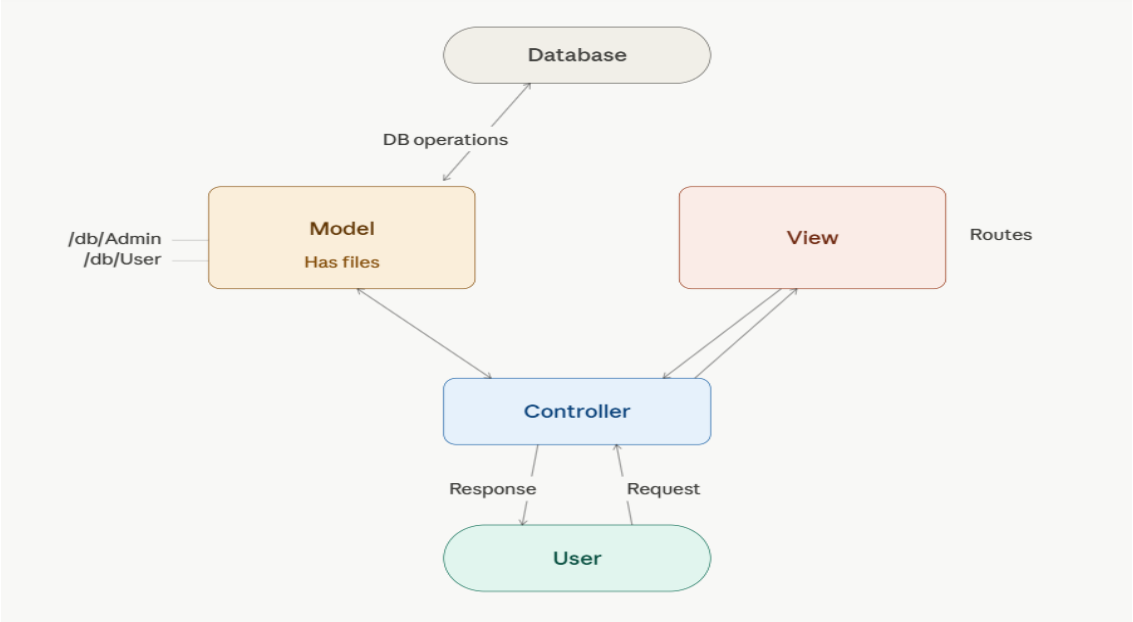
Customer Flow:

- Registration and Account — Create and manage account details securely with a profile picture and personal preferences.
- Browse and Search — Find recipes using search, category filters, and sorting options such as preparation time or difficulty level.
- View Recipe — Access full recipe details including ingredients, step-by-step instructions, cooking time, difficulty rating, and community reviews.
- Create and Share — Publish personal recipes with all required details and an uploaded dish photo for the community.
- Wishlist — Save favourite recipes to a personal wishlist for later access from the profile dashboard.
- Reviews — Leave star ratings and written feedback on recipes that have been tried.

Admin Flow:

- System Management — Oversee app performance, platform settings, and data security.
- User Management — Verify, approve, suspend, or manage user accounts as needed.
- Content Moderation — Review, edit, or remove recipes and user reviews that violate platform guidelines.
- Analytics — Monitor platform statistics and generate usage reports from the admin dashboard.

MVC PATTERN



The CookBook application follows the Model-View-Controller (MVC) architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.

Model Layer (Data Layer):

The Model layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database using those schemas. The models are implemented using Mongoose, which provides a schema-based solution to model application data for MongoDB. In CookBook, the Model layer defines schemas for Users, Admins, Recipes, Reviews, and Wishlists.

Controller Layer:

The Controller layer acts as an intermediary between the View (routes) and the Model. It receives incoming HTTP requests from the routes, processes the input — including validation and transformation — calls the appropriate Model methods to read or write data, and returns a structured JSON response to the client. In CookBook, separate controllers handle admin logic and user logic.

View Layer (Routing Layer):

In the context of a backend REST API, the View is implemented as the Routing layer, where various API endpoints are defined. These endpoints determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate Controller functions. In CookBook, separate route files handle admin routes and user routes.

Advantages of Using MVC in This Project:

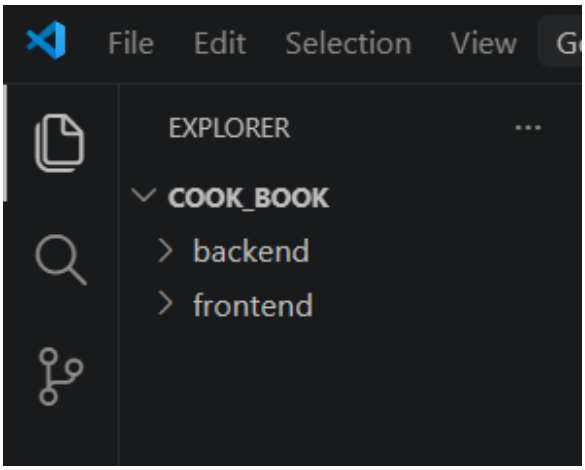
- Separation of Concerns: Each layer has a clearly defined responsibility, improving code readability and maintainability.
- Scalability: New features can be added easily by creating new routes, controllers, and models without affecting existing code.

- Reusability: Logic in controllers and models can be reused across multiple parts of the application.
- Testability: Each layer can be independently tested, especially the controllers and models, making debugging straightforward.
- Collaboration-Friendly: Multiple developers can work simultaneously on different layers without conflicts.

PROJECT SETUP AND CONFIGURATION

Creating the Project Folder:

- Create a new folder with your project name (e.g., CookBook).
- Inside that folder, create two sub-folders — name one Client and the other Server.
- Open the project folder in Visual Studio Code.



Client Setup (Installing React App):

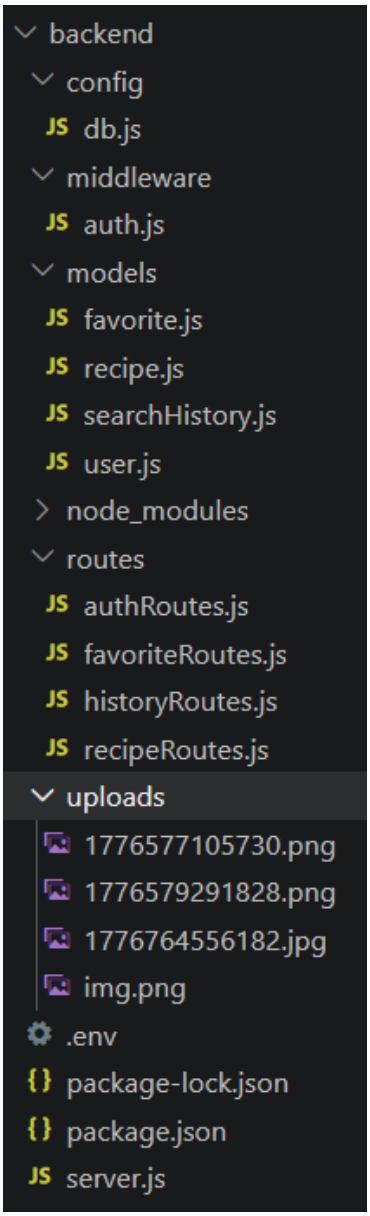
- Open the Client folder in the VS Code terminal.
- Run the following command: npm create vite@latest . -- --template react
- Select React as the framework from the given options.
- Select JavaScript as the variant from the given options.
- Navigate to the client folder: cd client
- Install all required packages: npm install
- Start the React development server: npm run dev

Server Setup (npm init)

- Open the Server folder in the VS Code terminal.
- Initialize npm: npm init -y
- Create the main server file: server.js
- Create the following folders inside Server: models, controllers, routes

BACKEND DEVELOPMENT

Backend Structure:



Controllers:

- **adminController.js:** Handles all admin-related business logic including login, managing users, moderating recipes, and viewing platform analytics.
- **userController.js:** Handles all user-related operations including registration, login, recipe CRUD operations, wishlist management, and review submission.

Db:

- **config.js:** Contains the database connection setup using Mongoose and environment variables stored in the .env file (e.g., MongoDB URI).

Middleware:

- **authMiddleware.js:** Middleware for authentication and authorization — validates JWT tokens and ensures protected routes are accessible only to authenticated users and admins.
- **upload.js:** Middleware for handling file uploads such as recipe images and user profile pictures, implemented using Multer.

Models:

- **Admin / Admins.js:** Schema and model for admin accounts (name, email, password, adminId).
- **User / Users.js:** Schema and model for user accounts (name, email, password, profile picture, userId).
- **User / Wishlist.js:** Schema and model for user wishlist items, referencing both the user ID and the recipe ID.
- **Recipe / Recipes.js:** Schema and model for recipe data including title, ingredients, steps, cooking time, difficulty, category, image URL, and author reference (userId).
- **Recipe / Reviews.js:** Schema and model for user reviews, including rating (1–5), comment text, timestamp, and references to the user and recipe.

Routes:

- **adminRoutes.js:** Defines all API endpoints for admin actions such as login, viewing all users, moderating recipes, and accessing dashboard analytics.
- **userRoutes.js:** Defines all API endpoints for user actions such as registration, login, browsing recipes, submitting reviews, managing wishlist, and personal recipe CRUD operations.

DATABASE DEVELOPMENT

1. Configure MongoDB

Install Mongoose in your Node.js project by running the following command in the Server directory:

npm install mongoose

Then create a database connection file and configure all Schemas and Models based on the ER diagram entities defined for the CookBook application.

Create Database Connection:

Ensure that the database is connected before performing any backend operations. The connection is established using Mongoose and environment variables from the .env file, and is called inside server.js before starting the Express server.

backend > config > JS db.js > connectDB

```
1   const mongoose = require("mongoose");
2
3   const connectDB = async () => {
4     try {
5       await mongoose.connect(process.env.MONGO_URI);
6       console.log("MongoDB Connected");
7     } catch (err) {
8       console.log(err);
9     }
10  };
11
12  module.exports = connectDB;
```

Create Schemas:

Configure the Schemas for the MongoDB database to store data in a structured pattern. Use the entities from the ER diagram to create all required schemas. The Schemas for this application are as follows:

1. Users.js → Schema/model for user accounts.

```
1   const mongoose = require("mongoose");
2
3   const userSchema = new mongoose.Schema(
4     {
5       name: String,
6       email: { type: String, unique: true },
7       password: String,
8     },
9     { timestamps: true },
10  );
11
12  module.exports = mongoose.model("User", userSchema);
13
```

2. recipe.js.

```
const mongoose = require("mongoose");

const recipeSchema = new mongoose.Schema({
  title: String,
  ingredients: [String],
  steps: [String],
  cookingTime: Number,
  cuisine: String,
  image: String, // ✅ ADD THIS
  createdBy: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
});

module.exports = mongoose.model("Recipe", recipeSchema);
```

3. favorite.js.

```
const mongoose = require("mongoose");

const favoriteSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
  recipeId: { type: mongoose.Schema.Types.ObjectId, ref: "Recipe" },
});

module.exports = mongoose.model("Favorite", favoriteSchema);
```

4. searchHistory.js.

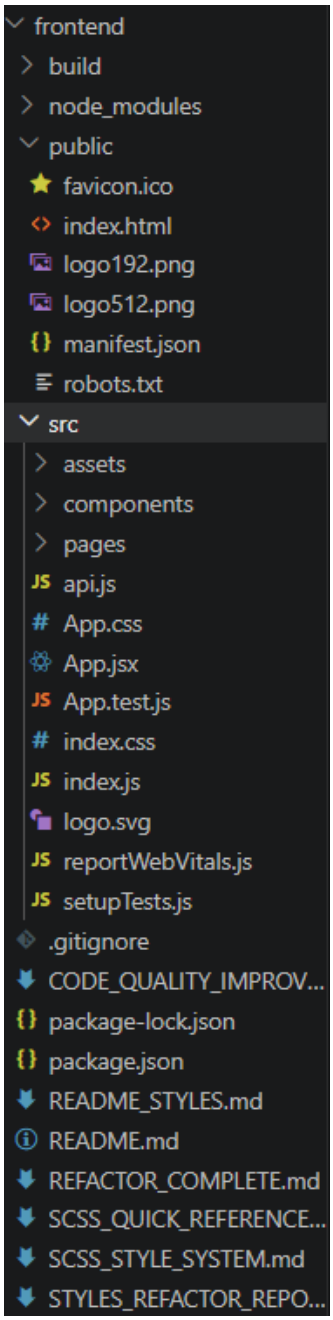
```
const mongoose = require("mongoose");

const historySchema = new mongoose.Schema(
  {
    userId: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
    searchQuery: String,
  },
  { timestamps: true },
);

module.exports = mongoose.model("SearchHistory", historySchema);
```

FRONT-END DEVELOPMENT

Frontend Structure:



Admin (src/Admin):

- **Ahome.jsx:** Admin dashboard/homepage displaying key platform statistics — total users, total recipes — and an overview bar chart.
- **Alogin.jsx:** Admin login page with email and password fields.
- **Anavbar.jsx:** Navigation bar for the admin panel with links to all admin sections.

- **Asignup.jsx:** Admin signup/registration page.
- **Users.jsx:** Admin panel to view all registered users with options to manage accounts.
- **AllRecipes.jsx:** Admin view for browsing all published recipes with options to moderate or remove content.

User (src/User):

- **Uhome.jsx:** User homepage/dashboard showing featured and recently added recipes.
- **Ulogin.jsx:** User login page with email and password fields.
- **Usignup.jsx:** User signup/registration page with name, email, and password fields.
- **Unavbar.jsx:** Navigation bar for the user side with links to Home, Browse, My Recipes, Wishlist, and Profile.
- **BrowseRecipes.jsx:** Displays all available recipes with search by name, filter by category, and sort by preparation time or difficulty.
- **RecipeDetail.jsx:** Full detail view of a single recipe including ingredients, step-by-step instructions, cooking time, difficulty, and a reviews section.
- **AddRecipe.jsx:** Form for users to create and publish a new recipe, including image upload.
- **MyRecipes.jsx:** Displays all recipes created by the currently logged-in user with options to edit or delete them.
- **Wishlist.jsx:** Displays all recipes saved to the user's personal wishlist/favourites collection.
- **WishListItem.jsx:** Reusable component for rendering a single saved recipe card in the wishlist page.
- **RecipeCard.jsx:** Reusable card component used to display a recipe thumbnail, title, and key info across various pages.

Components (Reusable Parts)

- **Footer.jsx:** Footer component used across all pages of the application.
- **Home.jsx:** Main landing page — the entry point for the app, visible to unauthenticated visitors with a call-to-action to register or browse recipes.
- **ui/:** Shared UI components such as buttons, modals, input fields, star rating widgets, and loading spinners used throughout the application.

CONCLUSION

The CookBook web application successfully delivers a centralized, user-friendly platform for recipe sharing, browsing, and community-driven culinary engagement. By leveraging the MERN stack — MongoDB, Express.js, React.js, and Node.js — and adhering to the MVC architectural

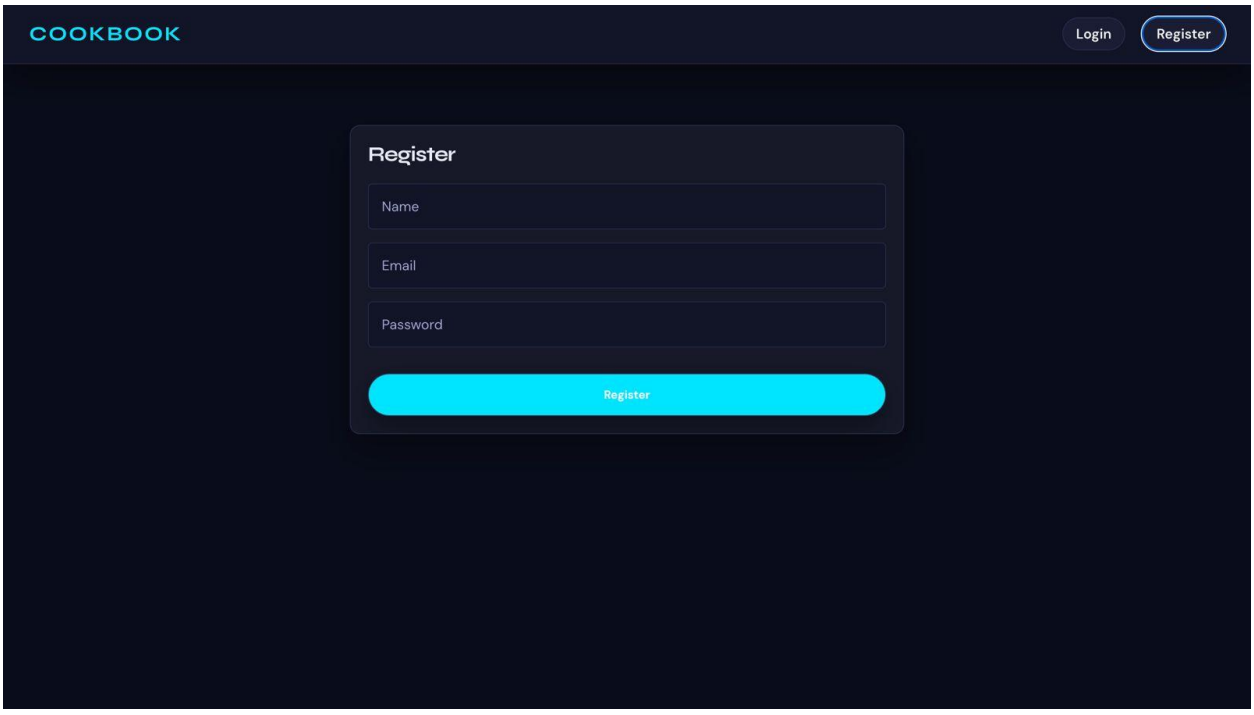
pattern, the application achieves a clean separation of concerns that makes it scalable, maintainable, and straightforward to extend with new features.

The platform serves two well-defined user roles — Users and Administrators — each with clearly scoped responsibilities and dedicated interfaces. Core features including recipe creation and browsing, category-based navigation, wishlist management, community reviews and ratings, and a comprehensive admin dashboard collectively deliver a complete and engaging culinary experience for the platform's community.

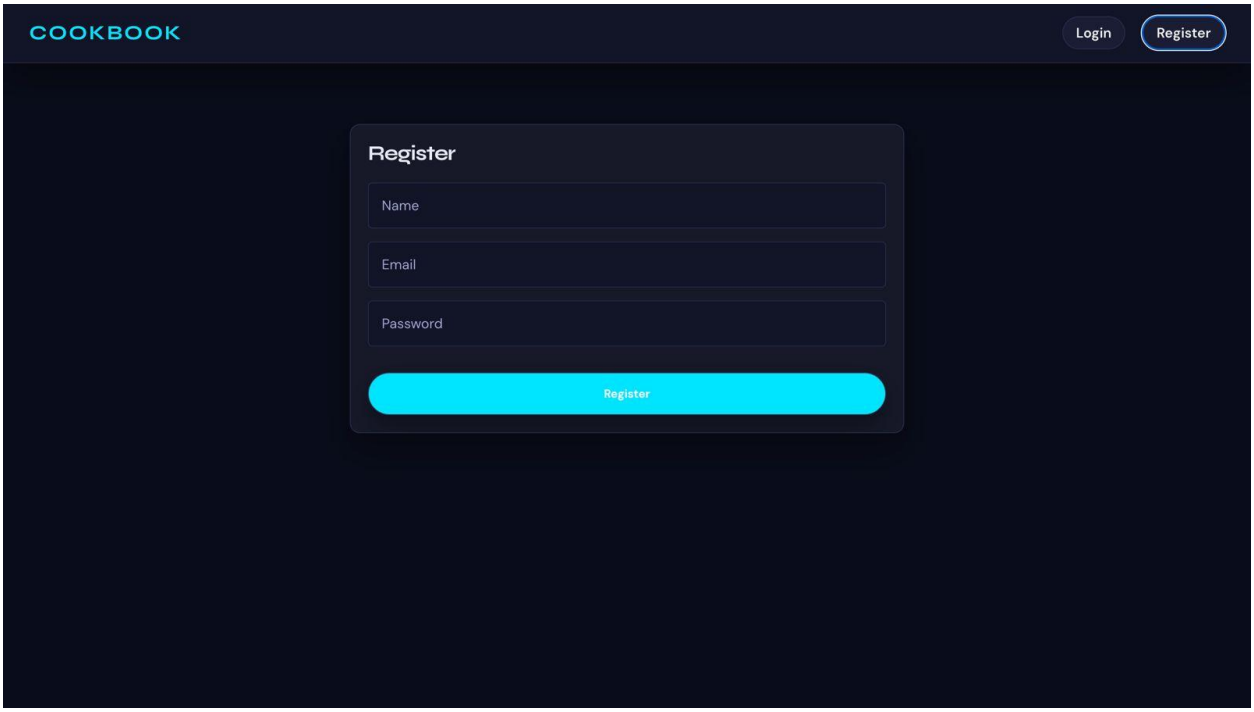
Through this project, the development team gained valuable hands-on experience in full-stack web development, RESTful API design, MongoDB schema modeling with Mongoose, JWT-based authentication, file upload handling with Multer, and responsive UI development using React.js and Vite. CookBook demonstrates how modern web technologies can be effectively combined to build real-world applications that bring value to food lovers, home cooks, and culinary learners alike.

Output ScreenShots:

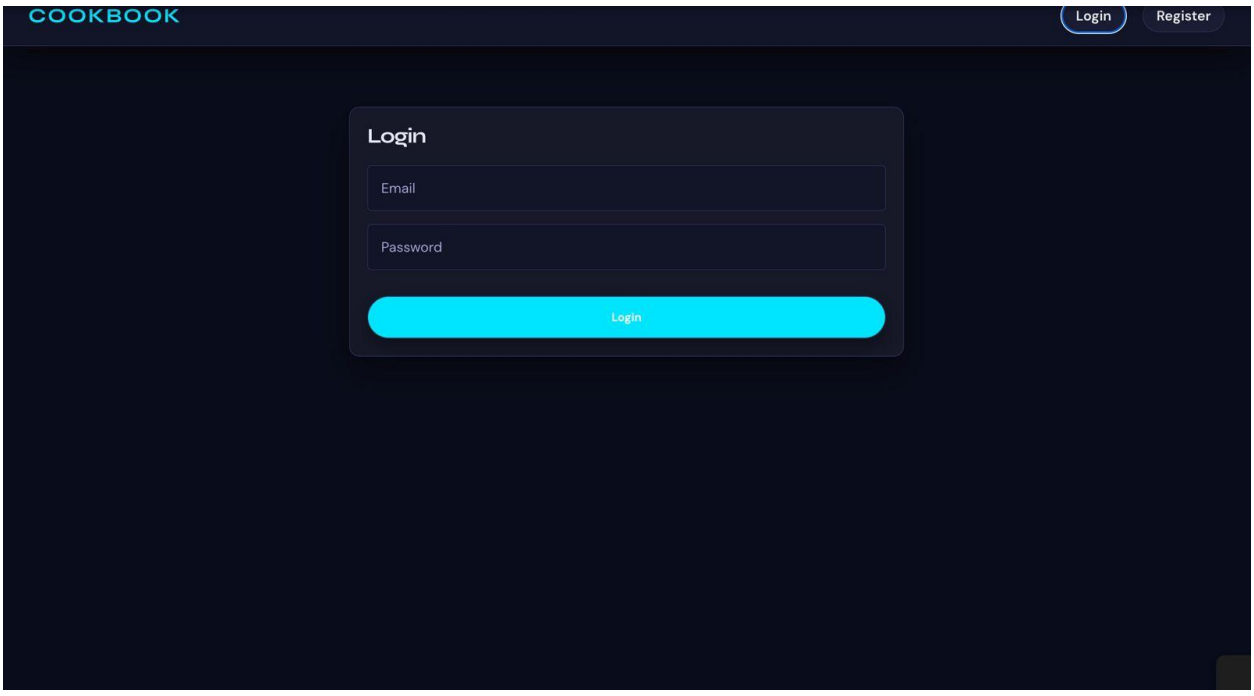
Landing Page:



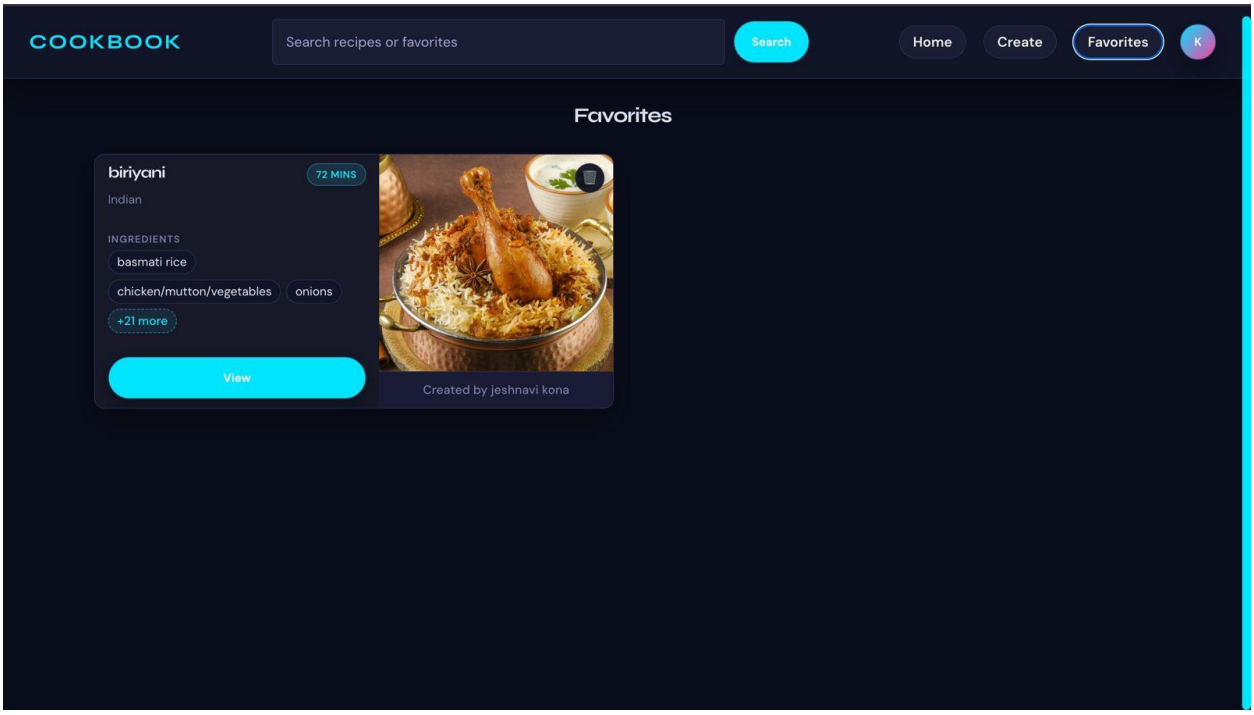
Register.jsx(User):



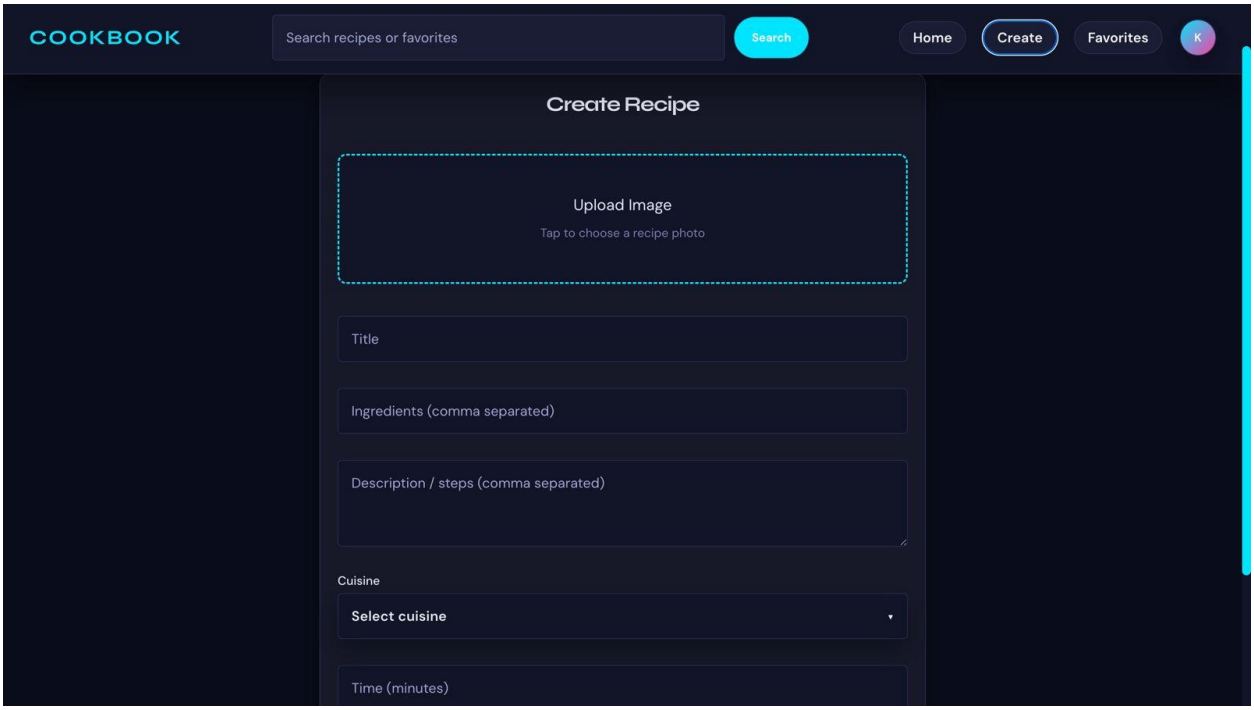
Login.jsx:



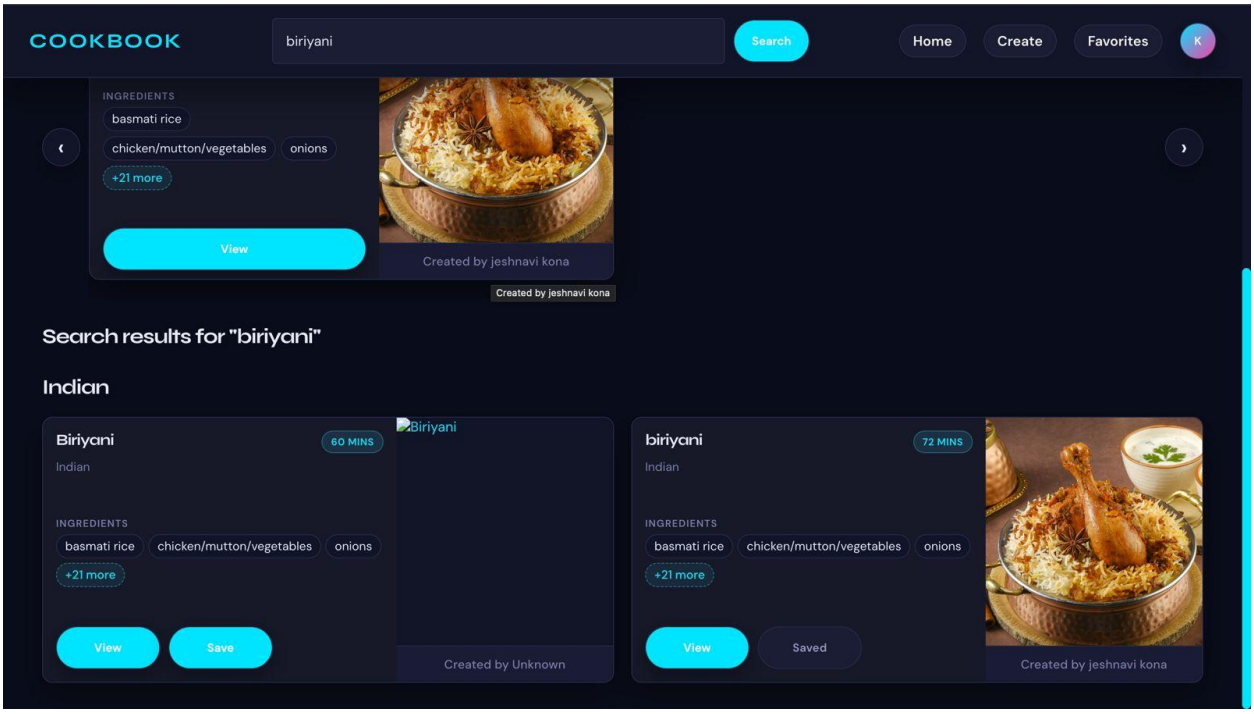
User Favourite Page:



Create Page:



Search Page:



Demo Video Link:
https://github.com/Jeshnavi19/cook_book/blob/main/projectOverview_video_cookbook.mp4

Code explanation video link:
https://github.com/Jeshnavi19/cook_book/blob/main/code_explanation_video_cookbook.mp4